

EEB3

BIG CLOCK

220 V Bulb 7-Segment Display

Driven by 4 x 8-Relay Modules

Arduino MEGA 2560 + DS3231 RTC

Power Architecture: Parallel 5V 3A Adapter with Opto-Isolation

Prepared by: **Taqi Abbas**

Location: European School of Brussels, Ixelles

Required library: RTCLib by Adafruit (v2.x)

Firmware: FW_VERSION "4.3-DIAG-2026-09-22"

1. SYSTEM OVERVIEW

What this clock does

A large wooden-frame wall clock for European School of Brussels, Ixelles. Each segment of each digit is an individual 220 V light bulb. Four 8-relay modules switch the bulbs (one module per digit). A DS3231 real-time clock module provides accurate timekeeping. An Arduino MEGA 2560 is the controller.

Display layout — HH:MM

Position	Module	Mega pins	Notes
Digit 1 — hours tens	Module 1	22–29	IN8 (pin 29) = colon dots
Digit 2 — hours units	Module 2	30–37	IN8 (pin 37) unused
Digit 3 — minutes tens	Module 3	38–45	IN8 (pin 45) unused
Digit 4 — minutes units	Module 4	46–53 (non-sequential – see 3c)	IN8 (pin 47) unused; IN3=pin48, IN6=pin51 (wiring-swap fix)

Relay-to-segment mapping (same for every module)

Each relay IN pin drives one bulb segment. The frame label e.g. 2.3 means digit 2, relay 3.

Relay / IN pin	Segment	Position on digit	Frame label
IN1	f	Top-left vertical	x . 1
IN2	a	Top horizontal	x . 2
IN3	b	Top-right vertical	x . 3
IN4	g	Middle horizontal	x . 4
IN5	e	Bottom-left vertical	x . 5
IN6	c	Bottom-right vertical	x . 6
IN7	d	Bottom horizontal	x . 7
IN8	—	Colon dots (Module 1 only)	1 . 8

2. HARDWARE LIST

Component	Qty
Arduino MEGA 2560 (Elegoo or genuine)	1
DS3231 RTC module with CR2032 coin cell	1
8-channel 5 V relay module — confirmed active-HIGH boards	4
External 5 V DC supply, minimum 3 A (regulated adapter)	1
USB-B cable (printer style) for programming	1
220 V E27 bulbs — one per segment, two for the colon	30
Mains-rated cable for bulb wiring	as needed
CR2032 spare coin cell (RTC backup battery)	1 spare
DuPont jumper wires, female-to-male	~50
Screw-terminal power rail for 5 V distribution	1

3. CONNECTIONS

3a. Power architecture (5V / 3A Parallel Supply with Opto-Isolation)

To provide enough power for all 4 relay boards (up to 2.5 A when active) and the Arduino Mega, a single external 5 V / 3 A DC regulated adapter is used. To isolate the Arduino from relay coil noise, remove the yellow jumpers between VCC and JD-VCC on the 3-pin headers of all relay boards. This runs the boards in opto-isolated mode.

Connection	From	To	Note
Main Adapter Power (+)	External 5V (+)	Mega 5V pin and all JD-VCC pins	Jumper on 3-pin relay headers MUST be removed
Main Adapter Ground (-)	External 5V (-)	Mega GND pin and all relay GND pins (3-pin header)	Common ground return for logic & coils
Optocoupler Power	Mega 5V pin	All 4 relay board VCC pins (10-pin header)	Powers opto-isolation diodes from Mega's quiet rail
RTC Power	Mega 5V pin	RTC VCC	Powers DS3231 (~2 mA)
RTC Ground	Mega GND	RTC GND	
RTC I2C Bus	Mega Pin 20 & 21	RTC SDA & SCL	Hardware I2C pins (Pin 20=SDA, Pin 21=SCL)

3b. RTC (DS3231) — 4 wires

RTC pin	Connect to	Notes
VCC	Mega 5V (POWER header)	Low current — Mega regulator is fine here
GND	Mega GND (any)	
SDA	Mega pin 20	Hardware I2C — only valid SDA pin on MEGA
SCL	Mega pin 21	Hardware I2C — only valid SCL pin on MEGA

Pins 20 and 21 are the ONLY hardware I2C pins on the Mega. Do not use any other pins.

3c. Signal wires — 32 wires total

All signal wires connect to the large double-row header at the top of the Mega (pins 22–53). Pin numbers are printed on the board.

Module 1 — HOURS TENS + colon on IN8

Board	Mega	Segment	Frame label
IN1	pin 22	f — top-left vertical	1.1
IN2	pin 23	a — top horizontal	1.2
IN3	pin 24	b — top-right vertical	1.3
IN4	pin 25	g — middle horizontal	1.4
IN5	pin 26	e — bottom-left vertical	1.5
IN6	pin 27	c — bottom-right vertical	1.6
IN7	pin 28	d — bottom horizontal	1.7
IN8	pin 29	colon dots	1.8

Module 2 — HOURS UNITS (IN8 unused)

Board	Mega	Segment	Frame label
IN1	pin 30	f — top-left vertical	2.1
IN2	pin 31	a — top horizontal	2.2
IN3	pin 32	b — top-right vertical	2.3
IN4	pin 33	g — middle horizontal	2.4
IN5	pin 34	e — bottom-left vertical	2.5
IN6	pin 35	c — bottom-right vertical	2.6
IN7	pin 36	d — bottom horizontal	2.7
IN8	pin 37	unused — leave empty	—

Module 3 — MINUTES TENS (IN8 unused)

Board	Mega	Segment	Frame label
IN1	pin 38	f — top-left vertical	3.1
IN2	pin 39	a — top horizontal	3.2
IN3	pin 40	b — top-right vertical	3.3
IN4	pin 41	g — middle horizontal	3.4
IN5	pin 42	e — bottom-left vertical	3.5
IN6	pin 43	c — bottom-right vertical	3.6
IN7	pin 44	d — bottom horizontal	3.7
IN8	pin 45	unused — leave empty	—

Module 4 — MINUTES UNITS (IN8 unused)

Board	Mega	Segment	Frame label
IN1	pin 53	f — top-left vertical	4.1
IN2	pin 52	a — top horizontal	4.2
IN3	pin 48	b — top-right vertical	4.3
IN4	pin 50	g — middle horizontal	4.4
IN5	pin 49	e — bottom-left vertical	4.5
IN6	pin 51	c — bottom-right vertical	4.6
IN7	pin 46	d — bottom horizontal	4.7
IN8	pin 47	unused — leave empty	—

3d. 220 V mains side

Wire each bulb through COM to NO so it is off at power-up.

Terminal	Connect to	Relay OFF	Relay ON
COM	Mains LIVE wire	—	—
NO	Bulb terminal A	Open — bulb off	Closed — bulb on
NC	Leave unconnected	Closed	Open

WARNING: 220 V wiring must be installed and verified by a qualified electrician. All mains wiring must be fused and enclosed. Keep mains and low-voltage wiring physically separated.

4. SETUP AND UPLOAD WORKFLOW

Arduino IDE settings (do once, never change)

Setting	Value
Board	Arduino Mega or Mega 2560
Processor	ATmega2560 (Mega 2560)
Port (Mac)	/dev/cu.usbmodem101 or /dev/cu.usbserial-XXXX
Port (Win)	COM3, COM4 ... check Device Manager
Library	RTCLib by Adafruit — install via Library Manager

Every time you upload — exact two-step workflow

The clock sets its own time automatically. No manual UTC entry, no typing, no extra tools to install. Your computer clock (NTP-synced) is the source of truth.

Step	What you do	What happens
1	Click Upload in Arduino IDE (as normal)	IDE compiles the sketch and uploads it to the Mega. Takes about 10–15 seconds. The board resets and prints READY on Serial.
2 – Mac	Double-click SetRTC_Mac.command in Finder	A Terminal window opens. Script connects to the Arduino, reads exact UTC from your Mac clock, sends it. RTC is set to within 1 second. Window shows confirmation then closes.
2 – Win	Double-click SetRTC_Windows.bat in File Explorer	A Command Prompt opens. Same process as Mac. RTC is set to within 1 second. Window shows confirmation then closes.

Important: Steps 1 and 2 overlap in time. You do NOT need to wait for Step 1 to finish before starting Step 2. Double-click the file while the upload bar is still moving — the script waits up to 30 seconds for the board to boot.

What if you skip Step 2?

If the script is not run within 8 seconds of the board booting, the Arduino automatically falls back to the compile-time stamp from your computer clock, converted to UTC. The clock will run but may be 10–20 seconds behind. For day-to-day use this is perfectly acceptable. Run Step 2 any time to correct it precisely.

Project files

File	Purpose	Platform
EEB3_Clock_Mega_Relay_PowerSaving.ino	Arduino sketch (in its own subfolder) — open and upload with IDE	Both
clock_diagnostics.html	Browser-based virtual clock + Web-Serial relay test UI (same subfolder as the .ino)	Both
SetRTC_Mac.command	Double-click after upload to set exact time	Mac only
SetRTC_Windows.bat	Double-click after upload to set exact time	Windows only
set_rtc.py	The actual time-setter — do not move or rename	Both
EEB3_Clock_Reference.pdf	This document	Both

First time on a new computer: the .command / .bat file installs the required pyserial library automatically. No manual pip install needed.

Coin cell replacement

The DS3231 uses a CR2032 cell. Typical life is 5–8 years. After replacement simply upload the sketch and run the time-setter script. The clock will be accurate within seconds.

5. DISPLAY BEHAVIOUR

Operating hours — relay rest cycle

All 32 relays are completely switched OFF outside operating hours. This gives the relay contacts a long daily rest, dramatically extending the lifespan of both the relays and the bulbs.

Time window	Display	All relays
Before CLOCK_ON	Nothing — completely blank	OFF
CLOCK_ON – CLOCK_OFF	Active time display	Normal operation
After CLOCK_OFF	Nothing — completely blank	OFF

Display logic

The display continuously shows HH:MM with a solid colon dots.

Condition	Seconds :00–:59	Colon State
Operating Hours Active	HH:MM	ON (Solid)
Night/Off Hours	Blank	OFF

6. DAYLIGHT-SAVING TIME LOGIC (BRUSSELS)

The RTC stores UTC permanently. Every second the code applies the EU DST rule mathematically — no lookup table, no manual intervention, valid for any future year.

Season	Offset	Starts	Ends
Summer — CEST	UTC + 2 h	Last Sunday of March, 01:00 UTC	Last Sunday of October, 01:00 UTC
Winter — CET	UTC + 1 h	Last Sunday of October, 01:00 UTC	Last Sunday of March, 01:00 UTC

How the calculation works

`lastSundayDay(year, month)` takes the 31st of the month, reads its day-of-week (0 = Sunday), and subtracts that from 31. Both March and October have 31 days so no special case is needed. This gives the correct last-Sunday date for any year.

If the EU ever abolishes DST, change only one line: replace `isEuSummerTime(utc) ? 2 : 1` with a fixed offset of 1 or 2.

7. SERIAL MONITOR DIAGNOSTICS

Open Serial Monitor at 9600 baud. Every time the display changes, diagnostic lines are printed:

```
[DISPLAY] Time -> 12:05 CEST+2
[FRAME] '12:05' colon=ON
```

Serial Monitor message	Meaning and action
[RTC] I2C 0x68: FAIL code=X	RTC not responding. Check SDA=20, SCL=21 wiring. Re-seat the module or check VCC/GND wires.
[RTC] lostPower: TRUE - NEEDS SYNC	CR2032 coin cell flat or missing. Replace cell and re-set UTC using Section 4 Steps 1 to 2.
No output at all after boot	Check port selection and baud rate (9600). Ensure power supply is active.
[ERROR] Bad year=XXXX	I2C glitch or corrupted RTC data. The code automatically displays dashes (----) on the clock frame. Sync the RTC.
Clock keeps re-printing the [BOOT] banner over and over	Watchdog is rebooting the Mega. Almost always caused by a hung I2C bus. Check SDA/SCL pullups and RTC connection.

7a. Wiring diagnostic protocol (D0 / D1 / R)

Beyond passive log-watching, the firmware accepts a small serial command set for actively tracing bulb-to-relay wiring one channel at a time, without waiting for the clock to reach a particular time of day.

Command	Effect
D1	Enter diagnostic mode: the clock display is paused and every relay is forced OFF.
D0	Exit diagnostic mode: normal HH:MM clock display resumes.
R<m><i><s>	While in diagnostic mode, fires one relay directly. m = module 0-3, i = relay index 0-7, s = 0 (off) or 1 (on). Example: R071 turns Module 0, IN8 ON — useful for locating the colon bulb.
T<10-digit unix UTC>	Syncs the RTC to an exact UTC timestamp, e.g. T1749736800. Works in or out of diagnostic mode; unchanged from earlier firmware.

These commands can be sent from the Arduino Serial Monitor (9600 baud, no line ending needed beyond CR/LF) but are normally driven automatically by the companion browser tool below.

7b. clock_diagnostics.html — browser wiring-test tool

The EEB3_Clock_Mega_Relay_PowerSaving folder includes clock_diagnostics.html, a self-contained browser page that combines a virtual on-screen clock with a live relay test panel driven over Web Serial. Open it in Chrome or Edge (Web Serial is not available in Safari or Firefox), click Connect, and select the Arduino's serial port. The page automatically sends D1 to pause the clock, then exposes a Test button for every relay on every module; each click sends the matching R<m><i><s> command so you can confirm exactly which bulb lights up for a given IN pin. Closing the page or clicking Disconnect sends D0 to resume the normal clock display.

8. COMMON ISSUES AND CHECKS

Symptom	Most likely cause	Check / Fix
Power LED flashes and instantly dies	Short circuit in wiring	Unplug relay board power wires first. Test bare Mega plus USB. Add wire groups back one at a time until short reappears.
Board not appearing in Mac/Windows port list	CH340 USB driver missing or bad cable	Download CH340 driver. Open Device Manager or System Information to verify. Use a high-quality USB-B data cable.
A segment never lights or is always on	Wire swapped or relay polarity wrong	Verify pin number against Section 3c. Use D1 + R<m><i><s> (Section 7a) or clock_diagnostics.html to fire that exact relay and confirm the physical wiring. Toggle RELAY_ACTIVE_LOW in code only if EVERY segment is inverted.
Time wrong by exactly 1 or 2 hours	RTC set to local time instead of UTC	Re-set RTC using the correct UTC value. See Section 4.
All bulbs off but Mega running	External 5V supply off or relay jumpers still on	Verify 5V 3A adapter is powered. Check that VCC/JD-VCC jumpers are REMOVED on all modules so coils get power from JD-VCC.
Mega reboots every 8 seconds	Watchdog triggered by stalled loop	Check RTC connections. High I2C noise can lock the bus.

9. RELIABILITY AND LONG-LIFE DESIGN NOTES

Opto-Isolation Power Routing

Removing the JD-VCC jumpers separates the relay coils' high-current circuit from the Arduino Mega's quiet digital supply. Inductive spikes and voltage sags from switching coils are confined to the external supply lines, preventing resets.

Daily relay rest — operating hours 08:00–17:00

All 32 relays are completely de-energised outside school hours. `allRelaysOff()` uses `writeRelay()` internally, so once the board is dark there are zero repeated clicks — only one transition at 08:00 and one at 17:00. That is ~15 hours of rest every day, dramatically extending relay and bulb lifespan.

Relay switching minimised

`writeRelay()` checks current state before driving the pin. A relay only clicks when its segment genuinely changes. Showing the same digit twice produces zero relay operations.

Colon stays on — never blinks

Blinking at 1 Hz would produce ~31 million operations per year on the colon relay alone. The colon stays steadily on during time display.

Hardware watchdog

`wdt_enable(WDTO_8S)` makes the microcontroller reboot itself if the main loop stops for more than 8 seconds, recovering from I2C hangs and unexpected lock-ups with no human intervention needed.

UTC-based timekeeping

The RTC stores UTC and is never adjusted for DST. The clock corrects itself on every DST boundary automatically, requiring no physical access and no buttons.

Defensive RTC reads

Any timestamp with a year outside 2024–2099 is silently discarded. This prevents corrupted I2C data from causing display errors.

10. FULL CODE LISTING

File: *EEB3_Clock_Mega_Relay_PowerSaving.ino* — FW_VERSION "4.3-DIAG-2026-09-22" — Board: *Arduino Mega or Mega 2560*, Processor: *ATmega2560*

```

/* =====
EEB3 SOLID-STATE CLOCK - 220V bulb 7-segment digits
RELEASE BUILD: September 22, 2026 (FW 4.2-BASIC)
-----

Board : Arduino MEGA 2560
Clock : DS3231 RTC (I2C, SDA=20, SCL=21 on MEGA)
Baud : 9600
-----

Stripped down to the essentials: accurate HH:MM time display, automatic
Brussels DST (CET/CEST) that recalculates every year forever (no hardcoded
dates), and a power-saving window that cuts the display outside school
hours. No period schedule, no boot self-test, no master-relay feature.
VERIFIED HARDWARE MAPPING (2026-09-22):
Module 1 (Hours Tens) : Pins 22-29 (IN1..IN8), IN8 = Colon
Module 2 (Hours Units) : Pins 30-37 (IN1..IN8), IN8 = Unused
Module 3 (Minutes Tens) : Pins 38-45 (IN1..IN8)
Module 4 (Minutes Units): Pins { 53, 52, 48, 50, 49, 51, 46, 47 }
IN3 = Pin 48 (Seg b - Top Right)
IN6 = Pin 51 (Seg c - Bottom Right)
SEGMENT TO RELAY MAPPING:
SEG2RELAY[7] order is a,b,c,d,e,f,g
a->1 (IN2), b->2 (IN3), c->5 (IN6), d->6 (IN7), e->4 (IN5), f->0 (IN1), g->3 (IN4)
const uint8_t SEG2RELAY[7] = { 1, 2, 5, 6, 4, 0, 3 };
POWER SAVING SCHEDULE:
CLOCK_ON_MINS = 8 * 60; // 08:00 AM
CLOCK_OFF_MINS = 17 * 60; // 05:00 PM (17:00)
TIME SYNC: send T<10-digit-unix-UTC>, e.g. T1749736800
WIRING DIAGNOSTICS (for tracing bulb/relay connections):
D1 -> enter diagnostic mode (pauses the clock, all relays off)
D0 -> exit diagnostic mode (resumes normal clock display)
R<m><i><s>-> while in diagnostic mode, set module m(0-3) index i(0-7)
to state s(0=off,1=on), e.g. R071 turns M0 IN8 ON
===== */
#include <Wire.h>
#include <RTClib.h>
#include <avr/wdt.h>
#define FW_VERSION "4.3-DIAG-2026-09-22"
#define SERIAL_BAUD 9600
#define SERIAL_WAIT_MS 8000UL
#define RELAY_ACTIVE_LOW false
// Power Saving active schedule: 08:00 AM to 05:00 PM
const int CLOCK_ON_MINS = 8 * 60; // 08:00 AM (480 minutes)
const int CLOCK_OFF_MINS = 17 * 60; // 05:00 PM (1020 minutes)
const uint8_t PINS[4][8] = {
{ 22, 23, 24, 25, 26, 27, 28, 29 }, // Module 1 (Hours Tens)
{ 30, 31, 32, 33, 34, 35, 36, 37 }, // Module 2 (Hours Units - IN8 unused)
{ 38, 39, 40, 41, 42, 43, 44, 45 }, // Module 3 (Minutes Tens)
{ 53, 52, 48, 50, 49, 51, 46, 47 } // Module 4 (Minutes Units - IN3=Pin48, IN6=Pin51)
};
const uint8_t COLON_RELAY_INDEX = 7;
// SEG2RELAY[seg] = relay index (a,b,c,d,e,f,g)
const uint8_t SEG2RELAY[7] = { 1, 2, 5, 6, 4, 0, 3 };
const char SEG_NAME[7] = {'a','b','c','d','e','f','g'};
bool relayState[4][8];
RTC_DS3231 rtc;
static bool rtcOk = false;
static bool diagMode = false;
/* =====
SERIAL TAG HELPERS
===== */
void tag(const __FlashStringHelper* t) {
Serial.print(F("")); Serial.print(t); Serial.print(F(" "));
}
void sep() { Serial.println(F("-----")); }

```

```

/* =====
SEGMENT FONT
bit0=a bit1=b bit2=c bit3=d bit4=e bit5=f bit6=g
===== */
byte segMask(char ch) {
  switch (ch) {
    case '0': return 0b0111111;
    case '1': return 0b0000110;
    case '2': return 0b1011011;
    case '3': return 0b1001111;
    case '4': return 0b1100110;
    case '5': return 0b1101101;
    case '6': return 0b1111011;
    case '7': return 0b0000111;
    case '8': return 0b1111111;
    case '9': return 0b1101111;
    case '-': return 0b1000000; // middle bar only (error indicator)
    case ' ':
    default: return 0b0000000;
  }
}
/* =====
RELAY ENGINE
===== */
inline void writeRelay(uint8_t module, uint8_t idx, bool on) {
  if (relayState[module][idx] == on) return;
  relayState[module][idx] = on;
  digitalWrite(PINS[module][idx], (on == RELAY_ACTIVE_LOW) ? LOW : HIGH);
  tag(F("RELAY"));
  Serial.print(F("M")); Serial.print(module);
  Serial.print(F(" IN")); Serial.print(idx + 1);
  Serial.print(F(" pin")); Serial.print(PINS[module][idx]);
  Serial.print(F(" -> ")); Serial.println(on ? F("ON") : F("OFF"));
}
void allRelaysOff() {
  tag(F("RELAY")); Serial.println(F("ALL OFF"));
  for (uint8_t m = 0; m < 4; m++)
    for (uint8_t i = 0; i < 8; i++)
      writeRelay(m, i, false);
}
void writeSegment(uint8_t module, uint8_t seg, bool on) {
  writeRelay(module, SEG2RELAY[seg], on);
}
void applyDigit(uint8_t module, char ch) {
  byte mask = segMask(ch);
  tag(F("DIGIT"));
  Serial.print(F("M")); Serial.print(module);
  Serial.print(F(" char=")); Serial.print(ch);
  Serial.print(F(" mask=0b"));
  for (int b = 6; b >= 0; b--) Serial.print((mask >> b) & 1);
  Serial.print(F(" segs: "));
  for (uint8_t s = 0; s < 7; s++)
    if (mask & (1 << s)) { Serial.print(SEG_NAME[s]); Serial.print(' '); }
  Serial.println();
  for (uint8_t seg = 0; seg < 7; seg++)
    writeSegment(module, seg, (bool)(mask & (1 << seg)));
}
void renderFrame(const char d[4], bool colonOn) {
  tag(F("FRAME"));
  Serial.print(F("")); Serial.print(d[0]); Serial.print(d[1]);
  Serial.print(F(":")); Serial.print(d[2]); Serial.print(d[3]);
  Serial.print(F(" colon=")); Serial.println(colonOn ? F("ON") : F("OFF"));
  for (uint8_t m = 0; m < 4; m++) applyDigit(m, d[m]);
  writeRelay(0, COLON_RELAY_INDEX, colonOn);
}
/* =====
DST / TIMEZONE (Brussels CET/CEST, EU rule - recalculated every year)
===== */
uint8_t lastSundayDay(uint16_t year, uint8_t month) {
  const uint8_t dim[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};

```

```

uint8_t d = dim[month];
if (month == 2 && (((year%4==0)&&(year%100!=0))||((year%400==0)))) d = 29;
return d - (uint8_t)DateTime(year, month, d, 0, 0, 0).dayOfTheWeek();
}
bool isEuSummerTime(const DateTime& u) {
uint8_t m = u.month();
if (m < 3 || m > 10) return false;
if (m > 3 && m < 10) return true;
uint8_t ls = lastSundayDay(u.year(), m);
if (m == 3) { if (u.day()>ls) return true; if (u.day()<ls) return false; return u.hour()>=1; }
if (u.day()<ls) return true; if (u.day()>ls) return false; return u.hour()<1;
}
bool isLocalSummerTime(const DateTime& l) {
uint8_t m = l.month();
if (m < 3 || m > 10) return false;
if (m > 3 && m < 10) return true;
uint8_t ls = lastSundayDay(l.year(), m);
if (m == 3) { if (l.day()>ls) return true; if (l.day()<ls) return false; return l.hour()>=2; }
if (l.day()<ls) return true; if (l.day()>ls) return false; return l.hour()<3;
}
DateTime utcToBrussels(const DateTime& utc) {
bool s = isEuSummerTime(utc);
tag(F("DST"));
Serial.print(s ? F("CEST+2 ") : F("CET+1 "));
DateTime local = utc + TimeSpan(0, s ? 2 : 1, 0, 0);
Serial.println(local.timestamp());
return local;
}
DateTime localToUtc(const DateTime& local) {
return local - TimeSpan(0, isLocalSummerTime(local) ? 2 : 1, 0, 0);
}
/* =====
RTC DIAGNOSTIC
===== */
void rtcDiagnostic() {
sep();
tag(F("RTC")); Serial.println(F("=== RTC Diagnostic ==="));
Wire.beginTransmission(0x68);
uint8_t err = Wire.endTransmission();
tag(F("RTC")); Serial.print(F("I2C 0x68: "));
if (err == 0) Serial.println(F("OK - device ACK"));
else { Serial.print(F("FAIL code=")); Serial.print(err); Serial.println(F(" CHECK SDA(20)/SCL(21)")); }
tag(F("RTC")); Serial.print(F("rtc.begin(): ")); Serial.println(rtcOk ? F("OK") : F("FAILED"));
if (!rtcOk) { sep(); return; }
bool lp = rtc.lostPower();
tag(F("RTC")); Serial.println(lp ? F("lostPower: TRUE - NEEDS SYNC") : F("lostPower: false - OK"));
DateTime now = rtc.now();
tag(F("RTC")); Serial.print(F("UTC raw: ")); Serial.print(now.timestamp());
Serial.print(F(" unix=")); Serial.println(now.unixtime());
tag(F("RTC")); Serial.print(F("Year check: "));
Serial.println((now.year()>=2024 && now.year()<=2099) ? F("PASS") : F("FAIL - sync needed!"));
tag(F("RTC")); Serial.print(F("Temp: ")); Serial.print(rtc.getTemperature(),1); Serial.println(F("C"));
DateTime local = utcToBrussels(now);
tag(F("RTC")); Serial.print(F("Brussels: "));
if (local.hour()<10)Serial.print('0'); Serial.print(local.hour());
Serial.print(':');
if (local.minute()<10)Serial.print('0'); Serial.println(local.minute());
int lm = (int)local.hour()*60+(int)local.minute();
tag(F("RTC")); Serial.print(F("Local mins=")); Serial.print(lm);
Serial.print(F(" window=")); Serial.print(CLOCK_ON_MINS);
Serial.print(F("-")); Serial.print(CLOCK_OFF_MINS);
Serial.println((lm>=CLOCK_ON_MINS&&lm<CLOCK_OFF_MINS) ? F(" ACTIVE") : F(" NIGHT MODE"));
sep();
}
/* =====
NON-BLOCKING SERIAL COMMAND PARSER
Send: T1749736800 (T + 10 digit UTC unix timestamp)
===== */
static char cmdBuf[16];
static uint8_t cmdPos = 0;

```

```

static bool inNightMode = false;
static unsigned long lastRenderedKey = 0xFFFFFFFFUL;
static uint32_t loopCount = 0;
static uint8_t heartbeat = 0;
void checkSerialCommands() {
while (Serial.available()) {
char c = (char)Serial.read();
if (c == '\n' || c == '\r') {
if (cmdPos > 0) {
cmdBuf[cmdPos] = '\0';
if (cmdBuf[0] == 'T' && strlen(cmdBuf) >= 11) {
unsigned long v = strtoul(cmdBuf + 1, nullptr, 10);
if (v > 1700000000UL) {
rtc.adjust(DateTime(v));
tag(F("SYNC")); Serial.print(F("RTC set UTC=")); Serial.println(DateTime(v).timestamp());
rtcDiagnostic();
}
} else if (cmdBuf[0] == 'D' && strlen(cmdBuf) == 2) {
diagMode = (cmdBuf[1] == '1');
allRelaysOff();
lastRenderedKey = 0xFFFFFFFFUL;
tag(F("DIAG")); Serial.println(diagMode ? F("ENTER") : F("EXIT"));
} else if (cmdBuf[0] == 'R' && strlen(cmdBuf) == 4 && diagMode) {
uint8_t m = cmdBuf[1] - '0', i = cmdBuf[2] - '0', s = cmdBuf[3] - '0';
if (m < 4 && i < 8) writeRelay(m, i, s != 0);
}
}
cmdPos = 0;
}
} else if (cmdPos < 15) {
cmdBuf[cmdPos++] = c;
}
}
}
/* =====
SETUP
===== */
void setup() {
for (uint8_t m=0;m<4;m++) for (uint8_t i=0;i<8;i++) {
uint8_t off = RELAY_ACTIVE_LOW ? HIGH : LOW;
digitalWrite(PINS[m][i], off);
pinMode(PINS[m][i], OUTPUT);
digitalWrite(PINS[m][i], off);
relayState[m][i] = false;
}
Serial.begin(SERIAL_BAUD);
Wire.begin();
Serial.println(F("READY"));
sep();
tag(F("BOOT")); Serial.print(F("EEB3 Clock FW=")); Serial.println(F(FW_VERSION));
tag(F("BOOT")); Serial.print(F("Compiled ")); Serial.print(F(__DATE__)); Serial.print(' ');
Serial.println(F(__TIME__));
tag(F("BOOT")); Serial.print(F("Window: ")); Serial.print(CLOCK_ON_MINS); Serial.print(F("-"));
Serial.println(CLOCK_OFF_MINS);
tag(F("BOOT")); Serial.println(RELAY_ACTIVE_LOW ? F("Relay: ACTIVE LOW") : F("Relay: ACTIVE HIGH"));
sep();
rtcOk = rtc.begin();
rtcDiagnostic();
tag(F("BOOT")); Serial.println(F("Waiting 8s for serial sync (T<unix>...)"));
unsigned long ws = millis(), ld = millis();
while (millis() - ws < SERIAL_WAIT_MS) {
wdt_reset();
if (millis() - ld >= 1000UL) {
ld = millis();
Serial.print('.');
}
}
checkSerialCommands();
}
Serial.println();
if (rtcOk && rtc.lostPower()) {
DateTime cl(F(__DATE__), F(__TIME__));

```

```

DateTime uf = localToUtc(cl);
rtc.adjust(uf);
tag(F("SYNC")); Serial.print(F("Fallback compile-time UTC=")); Serial.println(uf.timestamp());
}
tag(F("BOOT")); Serial.println(F("Watchdog WDT0_8S"));
wdt_enable(WDT0_8S);
tag(F("BOOT")); Serial.println(F("Entering main loop"));
sep();
}
/* =====
MAIN LOOP
===== */

void loop() {
  wdt_reset();
  loopCount++;
  checkSerialCommands();
  if (diagMode) return; // manual relay commands only; clock paused
  static unsigned long lastTick = 0;
  if (millis()-lastTick < 250UL) return;
  lastTick = millis();
  DateTime utc = rtc.now();
  if (++heartbeat >= 20) {
    heartbeat = 0;
    tag(F("LOOP"));
    Serial.print(F("tick #")); Serial.print(loopCount);
    Serial.print(F(" UTC=")); Serial.print(utc.timestamp());
    Serial.print(F(" temp=")); Serial.print(rtc.getTemperature(),1); Serial.println(F("C"));
  }
  // Year sanity - show dashes if RTC is corrupt
  if (utc.year()<2024 || utc.year()>2099) {
    tag(F("ERROR")); Serial.print(F("Bad year=")); Serial.println(utc.year());
    char bad[4]={'-','-','-','-'};
    renderFrame(bad, false);
    delay(1000);
    return;
  }
  bool summer = isEuSummerTime(utc);
  int8_t offset = summer ? 2 : 1;
  DateTime local = utc + TimeSpan(0, offset, 0, 0);
  uint8_t hh = local.hour();
  uint8_t mm = local.minute();
  int localMins = (int)hh*60+(int)mm;
  // Power saving: relays off outside the active window
  if (localMins<CLOCK_ON_MINS || localMins>=CLOCK_OFF_MINS) {
    if (!inNightMode) {
      inNightMode=true;
      lastRenderedKey=0xFFFFFFFFUL;
      allRelaysOff();
      tag(F("NIGHT"));
      Serial.print(F("Night at "));
      if(hh<10)Serial.print('0'); Serial.print(hh); Serial.print(':');
      if(mm<10)Serial.print('0'); Serial.println(mm);
    }
    return;
  }
  if (inNightMode) {
    inNightMode=false;
    tag(F("NIGHT")); Serial.println(F("Exited night mode"));
  }
  // Build display frame: HH:MM
  char frame[4];
  frame[0] = (hh<10) ? ' ' : (char)('0'+hh/10);
  frame[1] = (char)('0'+hh%10);
  frame[2] = (char)('0'+mm/10);
  frame[3] = (char)('0'+mm%10);
  unsigned long key =
  ((unsigned long)(uint8_t)frame[0]<<24) |
  ((unsigned long)(uint8_t)frame[1]<<16) |
  ((unsigned long)(uint8_t)frame[2]<< 8) |
  (unsigned long)(uint8_t)frame[3];

```

```
if (key != lastRenderedKey) {
  lastRenderedKey = key;
  tag(F("DISPLAY"));
  Serial.print(F("Time -> "));
  if(hh<10)Serial.print(' '); Serial.print(hh); Serial.print(':');
  if(mm<10)Serial.print('0'); Serial.print(mm);
  Serial.print(F(" ")); Serial.println(summer ? F("CEST+2") : F("CET+1"));
  renderFrame(frame, true);
  sep();
}
}
```

11. QUICK REFERENCE

All pin assignments

Function	Mega pins
RTC SDA / SCL	20, 21
Module 1 IN1–IN8 (hours tens + colon)	22 23 24 25 26 27 28 29
Module 2 IN1–IN8 (hours units)	30 31 32 33 34 35 36 37
Module 3 IN1–IN8 (minutes tens)	38 39 40 41 42 43 44 45
Module 4 IN1–IN8 (minutes units)	53 52 48 50 49 51 46 47

Configurable constants in code

Constant	Default	Change when
RELAY_ACTIVE_LOW	false	Confirmed hardware is active-HIGH; set true only if fitted with active-LOW relay boards
CLOCK_ON_MINS	480 (08:00)	Time when relays wake up (minutes from midnight, e.g. 8*60 = 480 for 08:00)
CLOCK_OFF_MINS	1020 (17:00)	Time when all relays go to sleep (minutes from midnight, e.g. 17*60 = 1020 for 17:00)
SERIAL_WAIT_MS	8000	Milliseconds to wait for the Python time-setter script at boot

Prepared by Taqi Abbas | European School of Brussels, Ixelles